



Chapter 13

Testing Strategies





Example 1: In 1963, USA, the rocket to Mars failed, losing \$ 10 million. The cause was a mistake in the FORTRAN program –

DO 5 I = 1, 3 was written as **DO 5 I = 1.3**

1963 年，美国飞往火星的火箭失败了，损失 1000 万，
由于 Fortran 中的循环语句写成了赋值语句





Example 2: [Washington Post, 1996]

Dallas, Aug. 23 — The captain of an American Airlines jet that crashed in Colombia last December entered an incorrect one-letter computer command that sent the plane into a mountain, the airline said today.

The crash killed all but four of the 163 people aboard.

American's investigators concluded that the captain of the Boeing 757 apparently thought he had entered the coordinates for the intended destination, Cali.

But on most South American aeronautical charts, the one-letter code for Cali is the same as the one for Bogota, 132 miles in the opposite direction.

The coordinates for Bogota directed the plane toward the mountain, according to a letter by Cecil Ewell, American's chief pilot and vice president for flight. The codes for Bogota and Cali are different in most computer databases, Ewell said.

达拉斯，8月23日——美国航空公司今天表示，去年12月在哥伦比亚坠毁的一架美国航空飞机的机长输入了一个错误的单字母计算机命令，导致飞机坠入了一座山。机上163人除4人外全部遇难。美国航空的调查人员得出结论，这架波音757飞机的机长显然以为他输入的是预定目的地卡利的坐标。但在大多数南美航海图上，卡利的一个字母代码与波哥大的相同，方向相反，相距132英里。





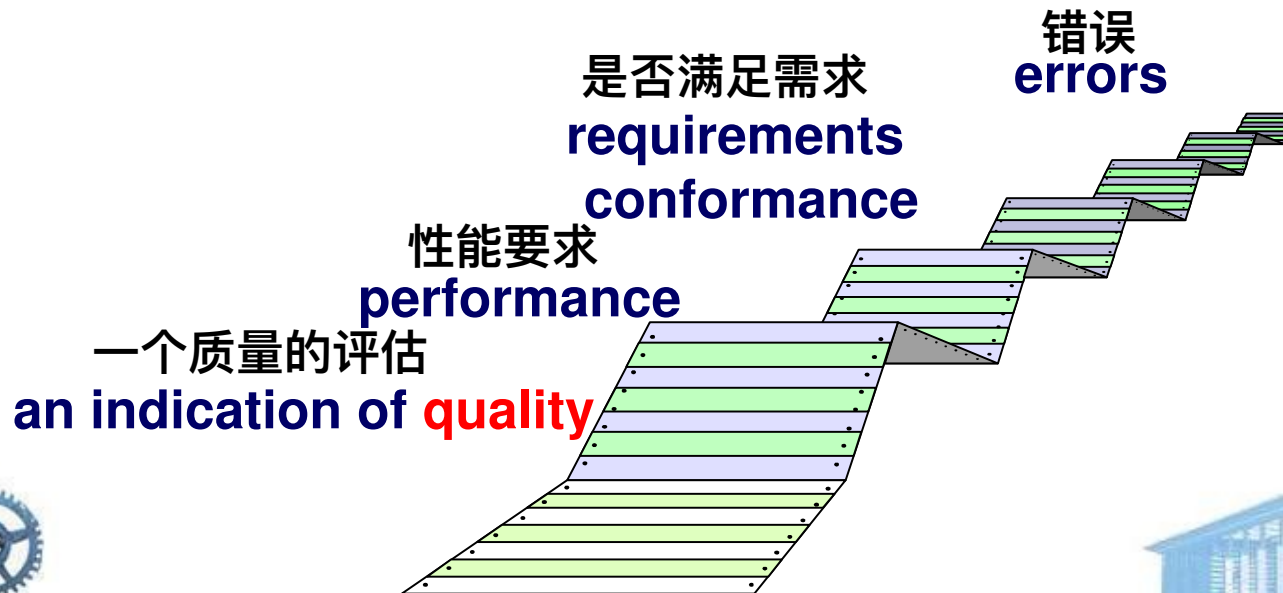
13.1 A Strategic Approach to Software Testing

- What is testing for? 测试是什么

Testing is the process of exercising a program with the specific intent of **finding errors** prior to delivery to the end user.

找到错误

- What testing shows? 测试能展示什么?





13.1 A Strategic Approach to Software Testing (测试策略)

- Testing **begins at the component level** and works outward **toward the integration** of the entire computer-based system. (测试从组件级别开始, 并向外扩展至整个系统的集成工作)
 - 单元测试 -> 集成测试
- Different testing techniques are appropriate at different points in time. (不同的测试技术适用于不同的时间点)
 - 单元测试多为白盒, 集成测试多为黑盒





13.1 A Strategic Approach to Software Testing (测试策略)

- The developer of the software conducts testing and may be assisted by independent test groups for large projects. (软件的开发人员来做测试, 并尽可能得到独立的测试小组的协助)
- **Testing (测试)** and **debugging (调试)** are different activities, but debugging must be accommodated in any testing strategy. (测试和调试是不同的活动, 但是调试必须包含在测试策略中)
 - Testing : 发现错误
 - Debugging : 改正错误





13.1 A Strategic Approach to Software Testing

- Verification (验证) and Validation (确认) (V & V)

Are we building the product right? (每个细节都是正确的吗)

Are we building the right product? (放在一起是正确的功能吗)



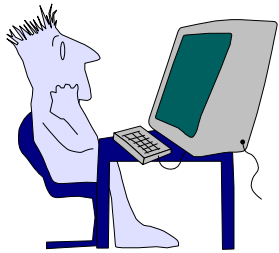
Software testing (软件测试) is only one element of SQA (Software Quality Assurance). (软件测试只是 SQA 软件质量保障的一个因素)

Quality must be built in to the development process, one **cannot** use testing to **add** quality **after** the fact. (质量管理必须内置到开发过程中，不能在事后使用测试来增加质量)



13.1 A Strategic Approach to Software Testing

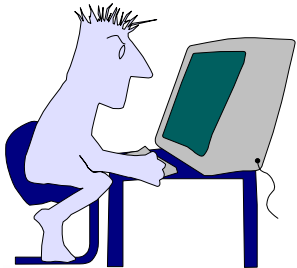
- Organizing for Software Testing (软件测试的组织)



Developer
编程人员

Understands the system, but will test “gently” and, is driven by “delivery”

理解系统，但会“温和”地进行测试，并以“交付”为导向



Independent Test Group(ITG)
独立的测试组

Must learn about the system, but will attempt to break it and, is driven by quality

必须了解系统，但会试图打破它，ITG 是由质量驱动的





13.1 A Strategic Approach to Software Testing

• Organizing for Software Testing (软件测试的组织)

WRONG	RIGHT
The developer should do no testing at all.	Unit + Integration. Then ITG gets in.
Software is tossed “over the wall” to ITG.	Work closely and remove errors.
Testers are not involved with the project until it is time for it to be tested.	Becomes involved during analysis and design and stays involved.

错误的：

开发人员不应该做任何测试。

软件被“翻墙”扔给 ITG。

直到项目需要测试时，测试人员才参与到项目中。参与分析和设计，并一直参与

正确的

单元 + 集成。然后 ITG 介入了

密切合作，消除错误。





13.1 A Strategic Approach to Software Testing

Exchange 2000 和 Windows 2000 的人员结构

	Exchange 2000	Windows 2000
项目经理	25 人	约 250 人
开发人员	140 人	约 1700 人
测试人员	350 人	约 3200 人
测试人员 / 开发人员	2: 5	1: 9





13.1 A Strategic Approach to Software Testing

- **For Traditional Software Architectures** （对传统的软件架构来说：）
 - **Unit Testing** （单元测试） - makes heavy use of testing techniques that exercise specific control paths to detect errors in each software *component* individually.
 - **Integration Testing** （集成测试） - focuses on issues associated with *verification* and program construction as components begin interacting with one another.
 - **Validation Testing** （确认测试） - provides assurance that the software *validation criteria* (established during requirements analysis) meets all functional, behavioral, and performance requirements. （确认与需求是不是一致的）
 - **System Testing** （系统测试） - verifies that all system elements mesh properly and that *overall* system function and performance has been achieved.





13.1 A Strategic Approach to Software Testing

- **For Object-Oriented Architectures (面向对象的架构)**

- **Unit Testing (单元测试)** - components being tested are **classes** not modules

- **Integration Testing (集成测试)** - as classes are integrated into the architecture regression tests are run to uncover **communication and collaboration** errors between objects

- **Systems Testing (系统测试)** - the system as a whole is tested to uncover **requirement** errors

- **When are we done? 什么时候结束测试**

Never!

When you run out of time
or you run out of money.





13.2 Strategic Issues

- Specify product requirements in a **quantifiable** manner before testing starts. (在测试开始前, 以可量化的方式指定产品需求)
- Specify testing objectives **explicitly**. (明确地指定测试目标)
- Identify categories of users for the software and develop a **profile** for each. (确定软件的用户类别, 并为每个用户开发一个概要文件)
- Develop a test plan that emphasizes **rapid cycle testing**. (指定一个强调快速循环测试的测试计划: 渐增的, 不是一蹴而就)
- Build **robust** software that is designed to test itself. (构建能够自我测试的健壮软件: 软件最好能抛出有用的异常信息)
- Use effective formal reviews as a filter **prior** to testing. (在测试之前使用有效的正式评审作为过滤: 不是运行代码, 而是通过讲解来评审代码)





13.2 Strategic Issues

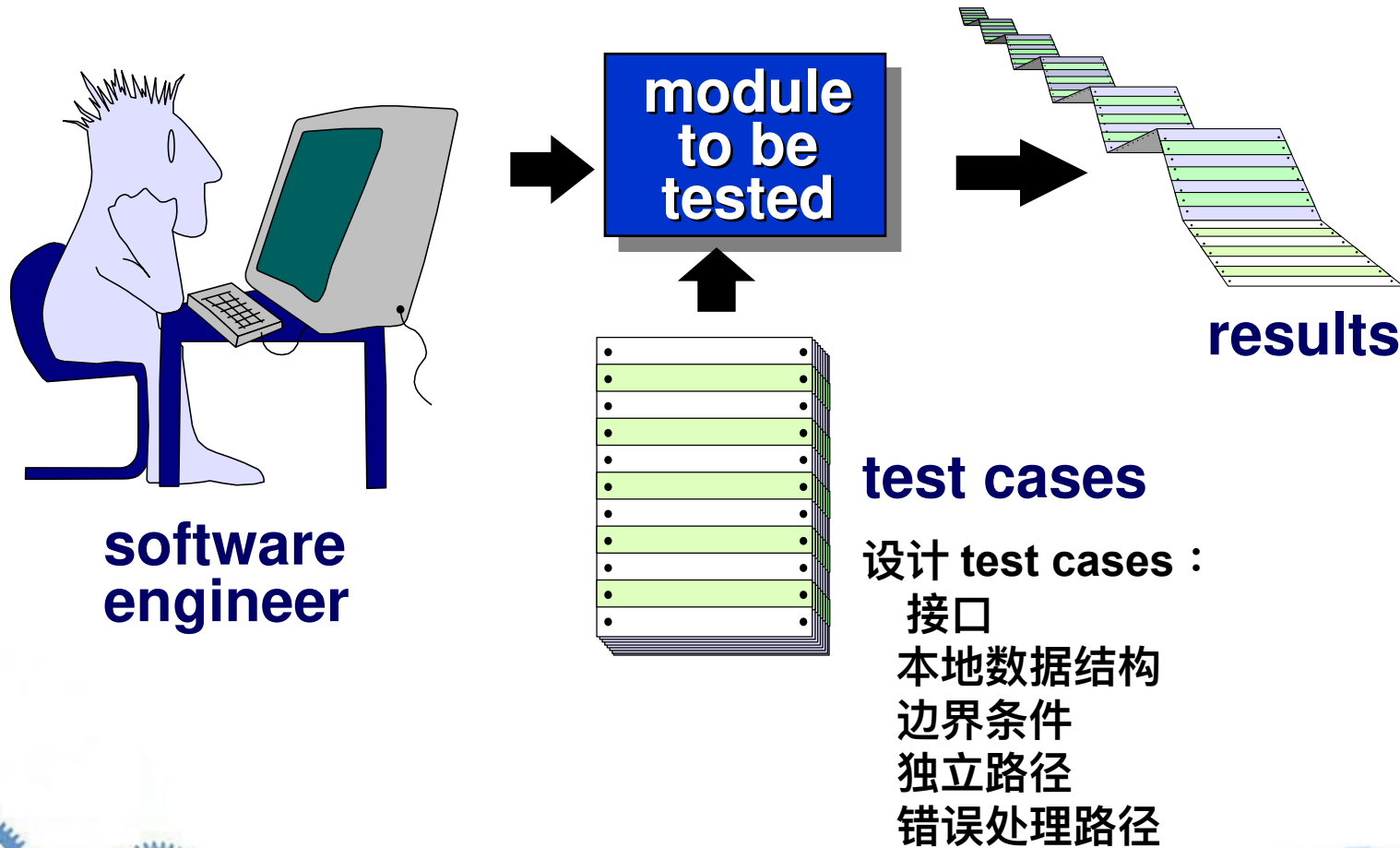
- Conduct ***formal technical reviews*** to assess the test strategy and test cases. (进行正式的技术评审，以评估测试策略和测试用例)
- Develop a ***continuous improvement*** approach for the testing process. (开发测试过程的持续改进方法)





13.3 For Conventional Software

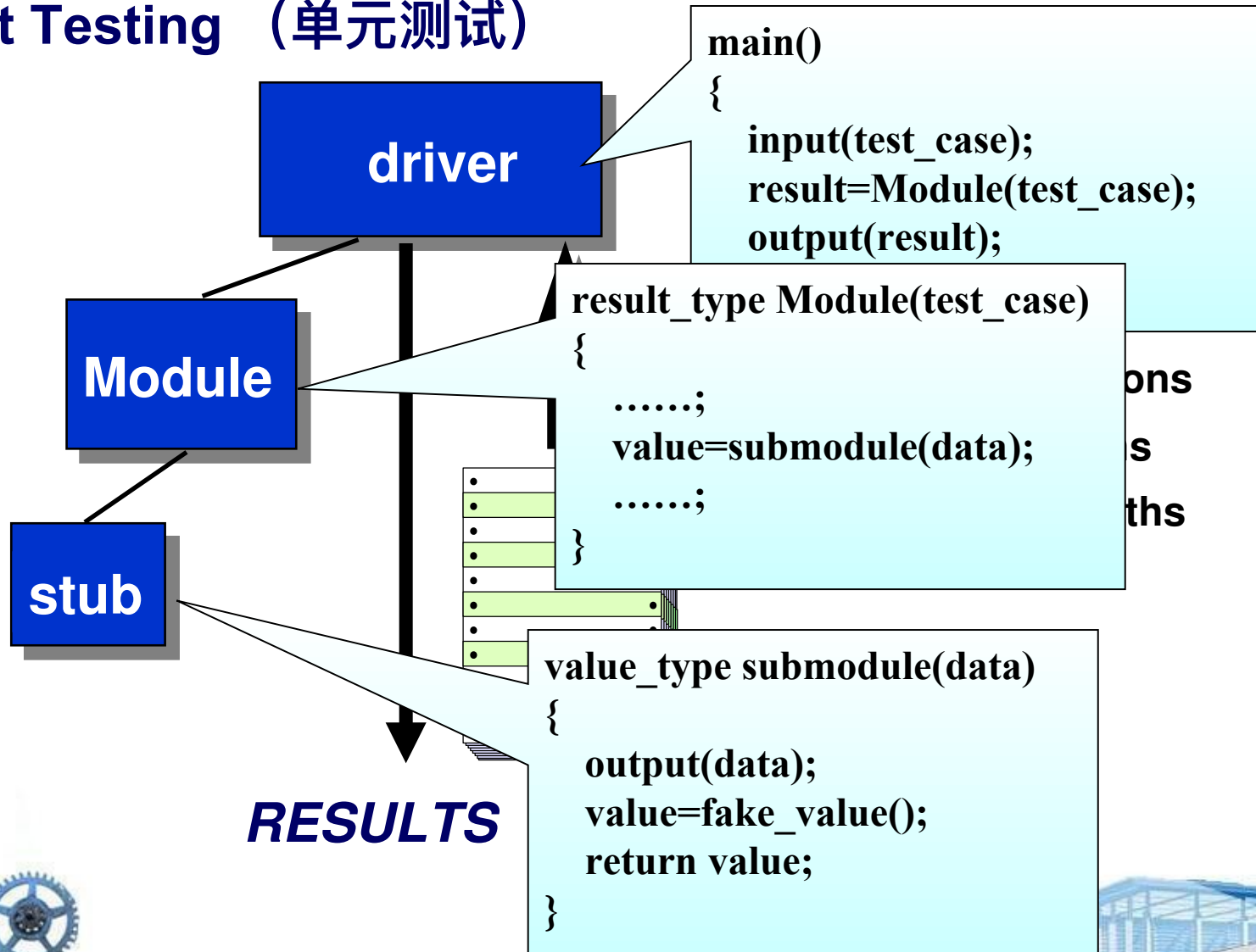
- Unit Testing (单元测试)





13.3 For Conventional Software

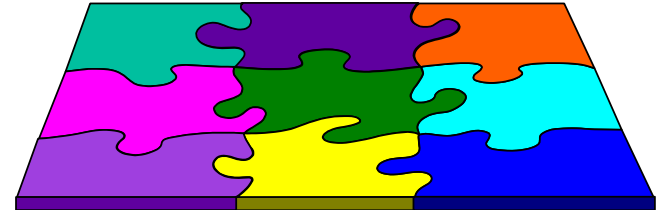
- Unit Testing (单元测试)



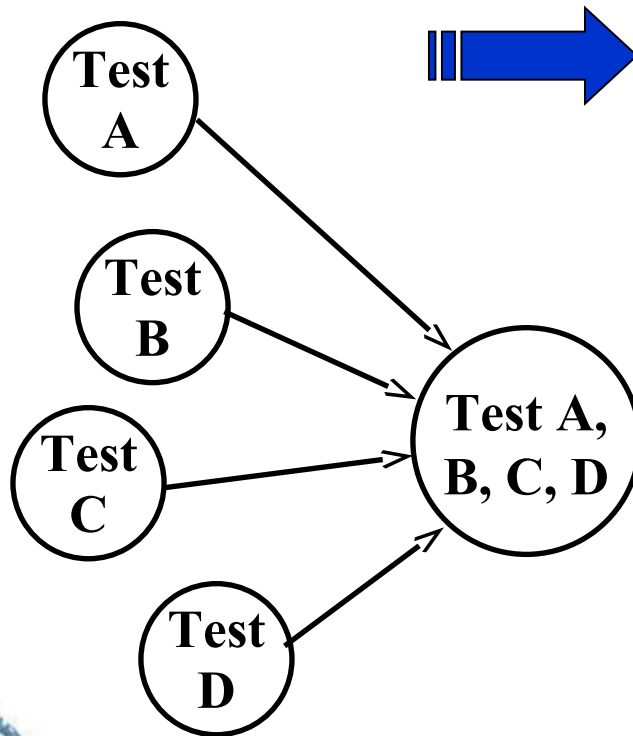


13.3 For Conventional Software

- Integration Testing (集成测试)



Big-bang testing (大锤测试)



Chaos !
(一团糟)



Isolation of causes

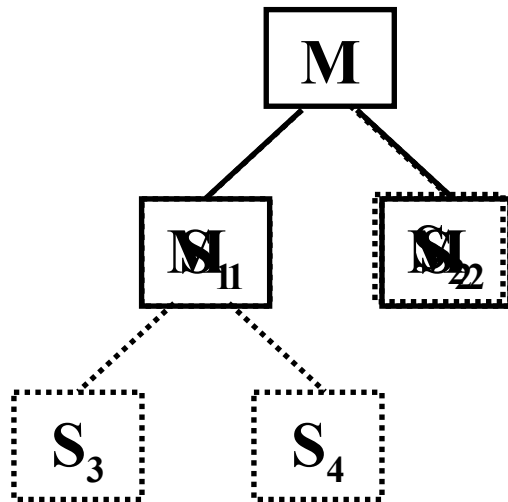
is complicated.
(不确定到底是
哪里的问题)





13.3 For Conventional Software

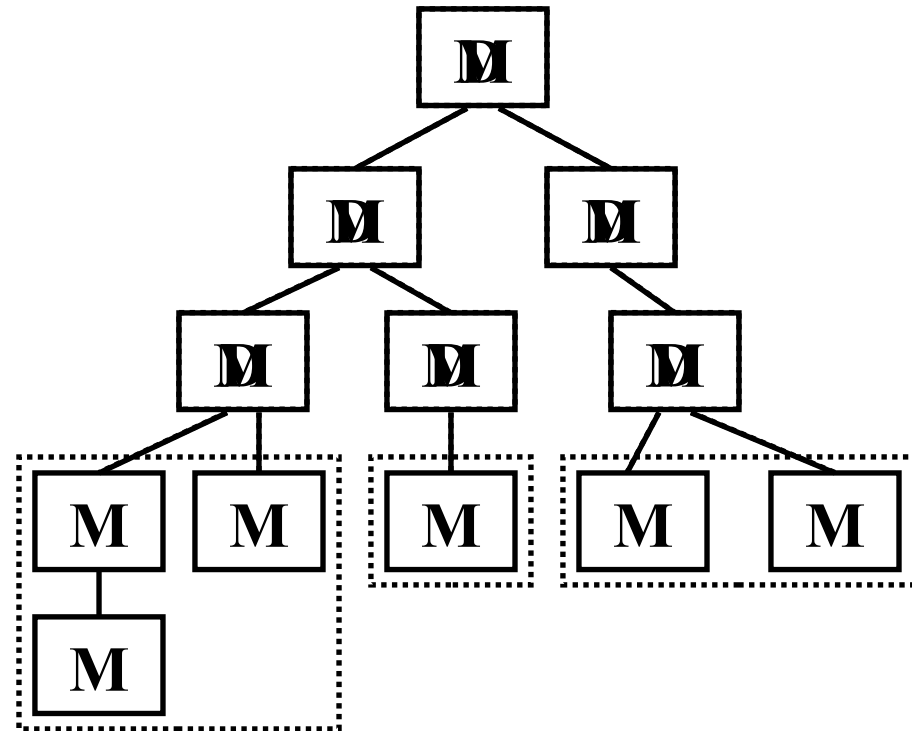
Top-down (从顶向下)



👍 verify major control or decision points early.
(能够尽早验证主要控制或决策的点)

👎 no significant data can flow upward. (到最后才测试真实数据)

Bottom-up (从底向上)



👍 使用真实的数据测试

👎 需要撰写控制模块



13.3 For Conventional Software

折中方案：

三明治法：两头测试，中间汇合

Regression testing（回归测试）：

to re-execute some subset of tests to ensure that **changes** have not propagated unintended side effects.

如果现在修改了，那么就需要重新执行测试的某个子集，以确保更改没有传播意外的副作用

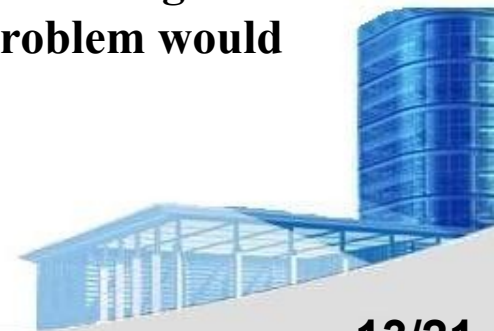




THE CONSEQUENCES OF NOT DOING REGRESSION TESTING

Seligman (1997) and Trager (1997) reported that 167,000 Californians were billed **\$667,000** for unwarranted local telephone calls because of a problem with software purchased from Northern Telecom. A similar problem was experienced by customers in New York City.

The problem stemmed from a fault in a **software upgrade** to the DMS-100 telephone switch. The fault caused the billing interface to use the wrong area code in telephone company offices that used more than one area code. As a result, local calls were billed as long-distance toll calls. When customers complained, the local telephone companies told their customers that the problem rested with the long-distance carrier; then the long-distance carrier sent the customers back to the local phone company! It took the local phone companies about a month to find and fix the cause of the problem. Had Northern Telecom performed **complete regression testing** on the software upgrade, including a check to see that area codes were reported properly, the billing problem would not have occurred.





不做回归测试的后果 - 案例

Seligman (1997) 和 Trager (1997) 报告说, 由于从北方电信购买的软件存在问题, 167,000 名加利福尼亚人被收取了 667,000 美元的无担保本地电话费用。纽约市的客户也遇到了类似的问题。

该问题源于 DMS-100 电话交换机的软件升级故障。该故障导致使用多个区号的电话公司办公室的计费接口使用错误的区号。结果, 本地电话被当作长途电话费计费。当客户投诉时, 当地电话公司告诉他们的客户, 问题在于长途运营商; 然后长途运营商把客户送回了当地的电话公司! 当地电话公司花了大约一个月的时间才找到并解决了问题的原因。如果 Northern Telecom 对软件升级执行完整的回归测试, 包括检查是否正确报告了区号, 就不会发生计费问题。

。

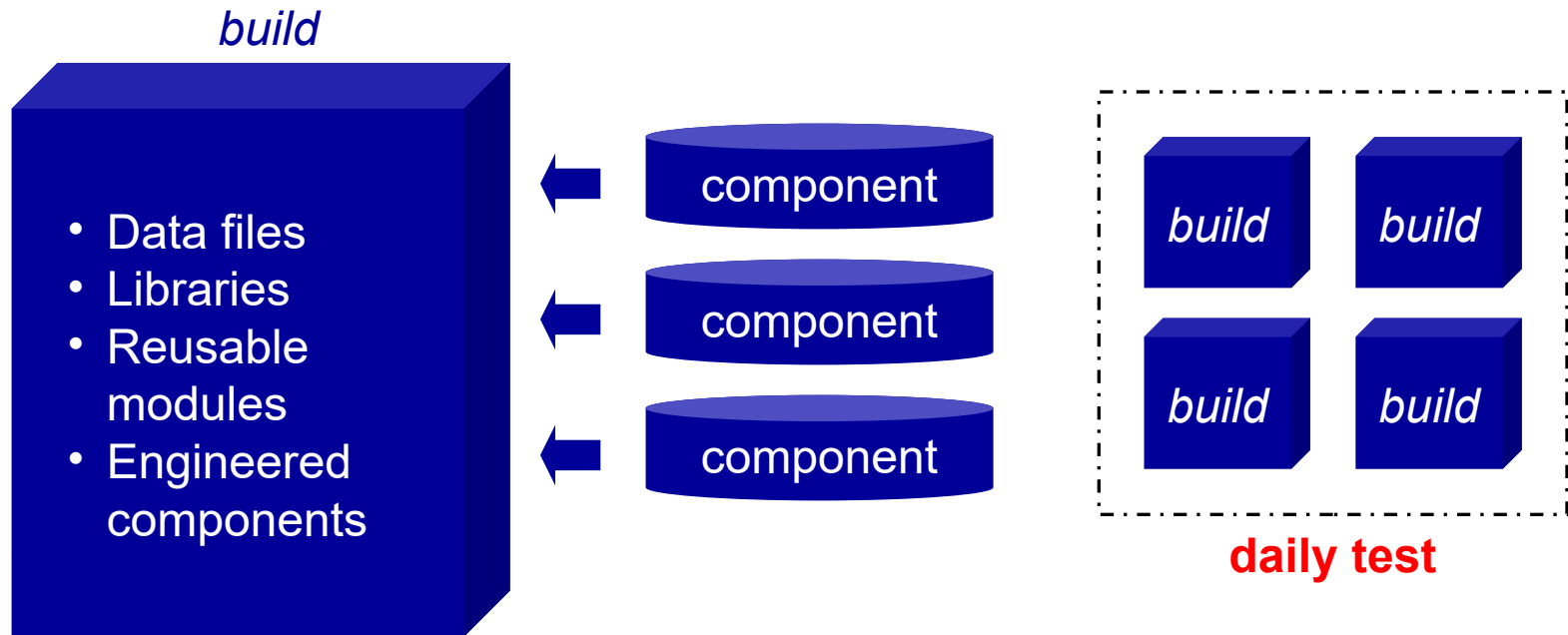




13.3 For Conventional Software

Smoke testing (Why is it called “smoke” testing?)

(冒烟测试：从硬件测试而来，如果新插入一个零件导致冒烟，说明新插进来的有问题)



Good for *time-critical* projects.





13.3 For Conventional Software

- **General Software Test Criteria** (通用软件测试准则)
 - **Interface integrity** (接口完整性) - internal and external module interfaces are tested as each module or cluster is added to the software (当每个模块或集群被添加到软件中时, 对内部和外部模块接口进行测试)
 - **Functional validity** (功能有效性) - test to uncover functional defects in the software (用于发现软件中功能缺陷的测试)
 - **Information content** (信息内容) - test for errors in local or global data structures (对具备或全局数据结构中的错误进行测试)
 - **Performance** (性能) - specified performance bounds are tested (测试性能指定的性能界限)





13.4 For Object-Oriented Software (对面向对象的软件而言)

- Unit Testing (单元测试) == Class Testing (类测试)
 - (之前的单元测试中, 一个单元是一个功能, 但是一个类可能包含多种功能)
-
- must not test operations in isolation (**hierarchy**) (不能孤立地测试, 要考虑层次结构)
 - driven by class **operations** and **state behavior**, not algorithmic detail and data flow across module interface (由类操作和状态行为驱动, 而不是由算法细节和跨模块接口的数据流驱动)





13.4 For Object-Oriented Software (对面向对象的软件而言)

- **Integration Testing (集成测试)**
 - **thread-based testing (基于线程的测试)** - testing all classes required to respond to one system input or event (测试所有需要响应的一个系统输入的一类或事件)
 - **use-based testing (基于使用的测试)** - begins by testing independent classes (classes that use very few server classes) first and then dependent classes that make use of them (首先测试独立类, 然后是使用它们的依赖类)
 - **cluster testing (集群测试)** - groups of collaborating classes are tested for interaction errors (测试协作类组的交互错误)





13.5 Validation Testing (确认测试)

*Software Requirements
Specification*
(确认软件需求是否被满足)

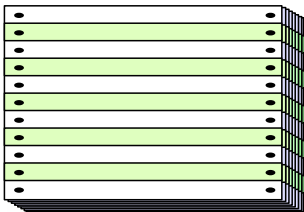
.....

>Validation Criteria

Based on

- use-case scenarios (用例场景)
- behavior model (行为模型)
- event flow diagram (事件流程图)

- **Configuration review (配置评审)** (audit)
- **Alpha Testing:** at the developer's site (开发者)
- **Beta Testing:** at the customer's site (客户)



**Deficiency
List (缺陷清单)**





13.6 System Testing (系统测试)

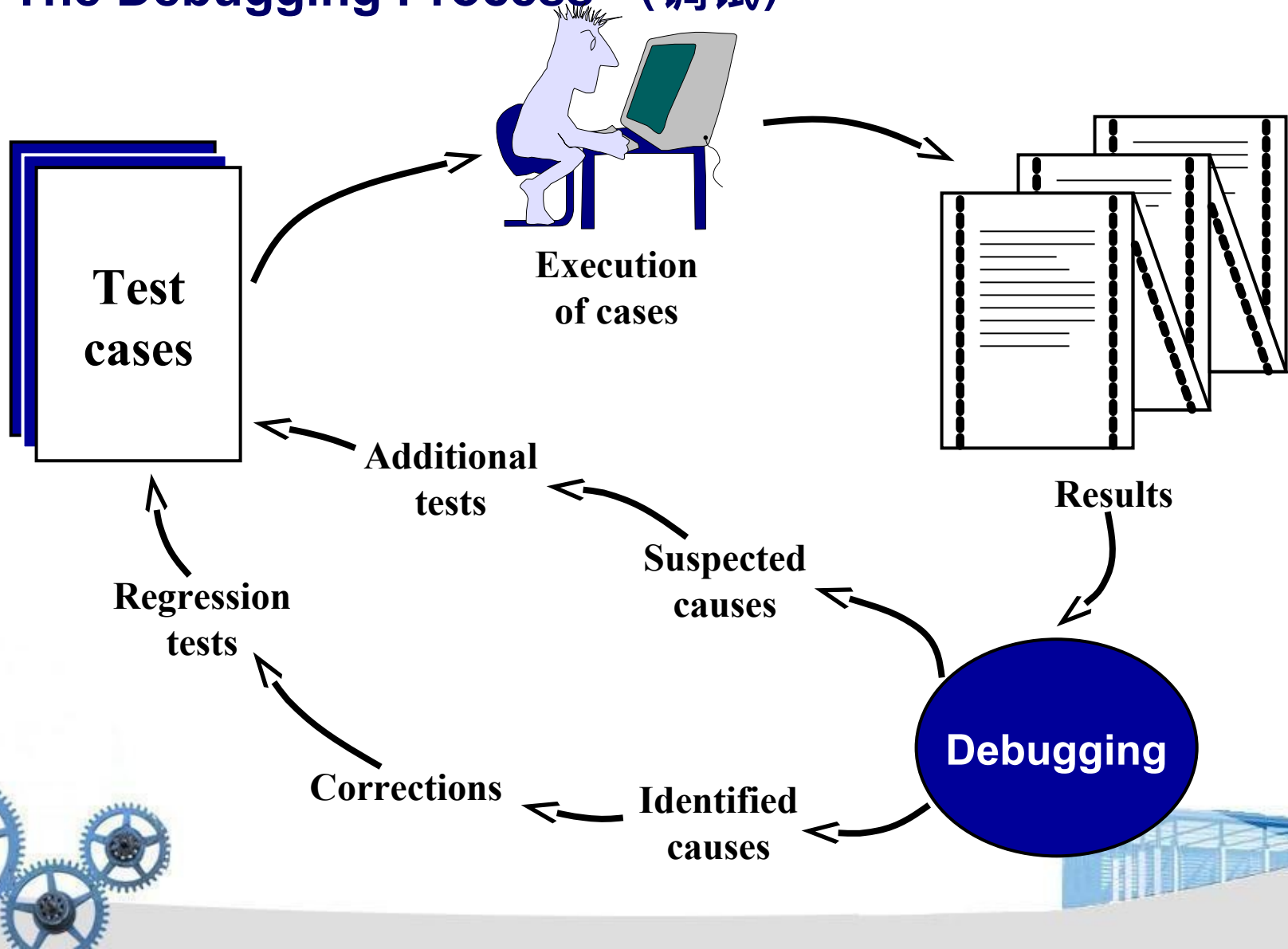
- **Recovery testing (恢复测试)** - force the software to *fail* (强制让软件失败)
- **Security testing (安全测试)** - verifies that system protection mechanism prevent *improper penetration* or data alteration (验证系统保护机制防止不当渗透或数据更改)
- **Stress testing (压力测试)** - program is checked to see how well it deals with *abnormal resource demands* (i.e., quantity, frequency, or volume) ---- How high can we crank this up before it fails? (检查程序, 看它如何处理异常的资源需求)
- **Performance testing (性能测试)** - designed to test the *run-time* performance of software, especially real-time software (用于测试软件, 特别是实时软件的运行时性能)





13.7 The Art of Debugging

- The Debugging Process (调试)

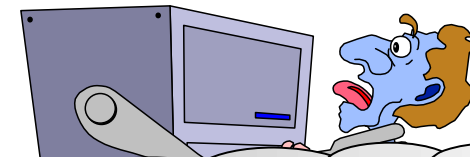




13.7 The Art of Debugging (调试策略)

- The Debugging Tactics

- brute force (强力测试)
- Backtracking (原路返回)
- Induction (归纳法)
- Deduction (演绎法)



Memory dump

Okay for small programs

Cause elimination

	Yes	No
What		
Where		
When		
How		

3W1H Table



13.7 The Art of Debugging

- **Bug Removal Considerations (bug 移除注意事项)**
 - Is the cause of the bug reproduced in another part of the program? (这个错误的原因是否在程序的其他部分重现?)
 - What “next bug” might be introduced by the fix that is being proposed? (提议的修复可能会引入下一个 bug 吗?)
 - What could have been done to prevent this bug in the first place? (一开始我们能做些什么来防止这个 bug 呢?)

